

Лекция 8. Алгоритм Эдмондса — Карпа.
Алгоритм Диница. Алгоритм Хопкрофта —
Карпа

1 Алгоритм Эдмондса — Карпа

Введение. Алгоритм Форда — Фалкерсона работает за $O(|E||f|)$, что слишком много, если пропускные способности могут быть большими. Также мы всё ещё не привели алгоритм, который бы корректно работал на графах с иррациональными пропускными способностями рёбер. Алгоритм Эдмондса — Карпа решает обе эти проблемы.

Алгоритм 1.1 (Эдмондса — Карпа). Пусть дана сеть $\mathcal{N} = (G = (V, E), c, s, t)$.

1. Положим все потоки равными нулю. Остаточная сеть изначально совпадает с исходной сетью.
2. В остаточной сети находим кратчайший путь из s в t (по количеству рёбер). Если такого пути нет, алгоритм завершается.
3. Через найденный увеличивающий путь пропускается максимально возможный поток. Его величина равна минимальной пропускной способности на этом пути.

Чтобы искать кратчайший по количеству рёбер увеличивающий путь, используется алгоритм BFS.

Теорема 1.2 (корректность). Алгоритм Эдмондса — Карпа корректно находит максимальный поток в сети.

Доказательство. Действительно, алгоритм завершается, когда в остаточной сети $\mathcal{N}_f$ нет пути из s в t . По теореме Форда — Фалкерсона из этого следует, что f максимален. $\square$

Теорема 1.3 (время работы). Алгоритм Эдмондса — Карпа работает за $O(|V||E|^2)$.

Доказательство. Чтобы доказать эту асимптотику, нужно доказать, что количество итераций (найденных путей) равно $O(|V||E|)$.

Лемма 1.4 (о неубывании расстояний). Пусть $\delta(v)$ — это кратчайшее расстояние (в количестве рёбер) от источника s до вершины v в текущей остаточной сети $\mathcal{N}_f$. Тогда после пропускания потока вдоль кратчайшего пути расстояние $\delta(v)$ ни для одной вершины не может уменьшиться.

Доказательство. От противного, пусть после пропускания потока мы получили новую остаточную сеть $\mathcal{N}_{f'}$, и для каких-то вершин расстояние от s уменьшилось. Выберем среди них вершину v , которая в новой сети оказалась ближе всего к s (то есть $\delta'(v) < \delta(v)$ и $\delta'(v)$ минимально). Пусть вершина u — вершина, предшествующая v на новом кратчайшем пути. Тогда $\delta'(v) = \delta'(u) + 1$. Так как v выбрана ближайшей вершиной к s , у которой расстояние уменьшилось, у u оно не уменьшилось: $\delta'(u) \geq \delta(u)$.

Могло ли ребро $u \rightarrow v$ существовать в старой сети? Нет, иначе было бы $\delta(v) \leq \delta(u) + 1 \leq \delta'(u) + 1 = \delta'(v)$, что противоречит тому, что $\delta(v)$ уменьшилось. Значит, ребро $u \rightarrow v$ появилось только что.

Заметим, что в остаточной сети новые рёбра могут появляться только как обратные. То есть на этом шаге мы пропустили поток по ребру $v \rightarrow u$. Но мы пускаем поток только по кратчайшим путям, значит, в старой сети $\delta(u) = \delta(v) + 1$. Таким образом,

$$\delta'(v) = \delta'(u) + 1 \geq \delta(u) + 1 = \delta(v) + 2$$

Следовательно, $\delta'(v) \geq \delta(v) + 2$, но по предположению $\delta'(v) < \delta(v)$. Противоречие, значит, расстояния в остаточной сети могут только увеличиваться. $\square$

На каждом шаге (на каждом найденном пути) хотя бы одно ребро пути заполняется полностью. Такое ребро назовём *критическим*. После пропускания потока критическое ребро $u \rightarrow v$ исчезает из остаточной сети (его остаточная пропускная способность становится равной 0). Оценим, сколько раз одно и то же ребро $u \rightarrow v$ может стать критическим за всё время работы алгоритма.

Когда ребро $u \rightarrow v$ становится критическим, оно лежит на кратчайшем пути, а значит, $\delta(v) = \delta(u) + 1$. Это ребро исчезает из остаточной сети. Чтобы оно могло появиться снова и в будущем снова стать критическим, мы должны когда-нибудь пропустить поток в обратном направлении, то есть по ребру $v \rightarrow u$. В тот момент, когда мы будем пропускать поток по $v \rightarrow u$, новые расстояния δ' будут рассчитываться по правилу кратчайшего пути, то есть $\delta'(u) = \delta'(v) + 1$. По доказанной лемме, $\delta'(v) \geq \delta(v)$. Подставим:

$$\delta'(u) = \delta'(v) + 1 \geq \delta(v) + 1 = (\delta(u) + 1) + 1 = \delta(u) + 2$$

Следовательно, между двумя моментами, когда ребро $u \rightarrow v$ становится критическим, расстояние от s до u должно увеличиться хотя бы на 2. Максимальное расстояние $\delta(u)$ не превышает $(|V| - 1)$ (иначе пути нет). Следовательно, каждое направленное ребро $u \rightarrow v$ может побывать критическим $|V|/2$ раз.

В исходном графе $|E|$ рёбер. В остаточной сети может быть максимум $2|E|$ направленных рёбер (прямые и обратные). Каждое из $2|E|$ рёбер может стать критическим не более $|V|/2$ раз. Значит, суммарное количество критических рёбер за весь алгоритм не более $2|E| \cdot (|V|/2) = |V||E|$. Так как на каждом пути есть хотя бы одно критическое ребро, общее количество итераций алгоритма не более $|V||E|$. Каждая итерация — это BFS, который работает за $O(|E|)$. В итоге получаем $O(|V||E|) \cdot O(|E|) = O(|V||E|^2)$ — что и требовалось доказать. $\square$

Псевдокод. Приведём реализацию алгоритма Эдмондса — Карпа.

```

1 int INF = 2e9;
2
3 struct EdmondsCarp {
4     struct Edge {
5         int to, rev, cap, flow;
6         Edge(int to, int rev, int cap, int flow) :
7             to(to), rev(rev), cap(cap), flow(flow) {}
8     };
9
10    int n;
11    vector<vector<Edge>> g;
12
13    EdmondsCarp(int n) : n(n), g(n) {}
14
15    void add_edge(int u, int v, int cap) {
16        g[u].emplace_back(v, (int)g[v].size(), cap, 0);
17        g[v].emplace_back(u, (int)g[u].size() - 1, 0, 0);
18    }
19
20    bool bfs(int s, int t, vector<pair<int, int>>& parent) {
21        parent.assign(n, {-1, -1});
22        queue<int> q;
23        q.emplace(s);
24        parent[s] = {-2, -2};
25        while (!q.empty()) {
26            int u = q.front();
27            q.pop();
28            for (int i = 0; i < g[u].size(); ++i) {
29                Edge& e = g[u][i];
30                if (parent[e.to].first == -1 && e.cap - e.flow > 0) {
31                    q.push(e.to);
32                    parent[e.to] = {u, i};
33                    if (e.to == t) {
```

```

34         return true;
35     }
36 }
37 }
38 }
39 return false;
40 }
41
42 int max_flow(int s, int t) {
43     int ans_flow = 0;
44     vector<pair<int, int>> parent;
45     while (bfs(s, t, parent)) {
46         int path_flow = INF;
47         for (int v = t; v != s; v = parent[v].first) {
48             int u = parent[v].first;
49             int edge_idx = parent[v].second;
50             path_flow = min(path_flow, g[u][edge_idx].cap - g[u][edge_idx].flow);
51         }
52         ans_flow += path_flow;
53         for (int v = t; v != s; v = parent[v].first) {
54             int u = parent[v].first;
55             int edge_idx = parent[v].second;
56             g[u][edge_idx].flow += path_flow;
57             int rev_idx = g[u][edge_idx].rev;
58             g[v][rev_idx].flow -= path_flow;
59         }
60     }
61     return ans_flow;
62 }
63 };

```

Заключение. Алгоритм Эдмондса — Карпа получается заменой DFS на BFS в алгоритме Форда — Фалкерсона. Он решает задачу поиска максимального потока в сети за $O(|V||E|^2)$.

2 Алгоритм Диница

Введение. Как вы уже заметили, алгоритмы поиска потока достаточно дорогие по времени. Алгоритм Эдмондса — Карпа на плотных графах, когда $|E| \sim |V|^2$, может вырождаться в $O(|V|^5)$, что достаточно много. Конечно, это далеко не наилучший существующий алгоритм поиска максимального потока, но чтобы придумать нечто лучшее, нам придётся немного отойти от оригинального подхода Форда — Фалкерсона (поиска одиночных путей) и построить новую концепцию (слоистые сети), на которой можно будет придумать алгоритм быстрее.

Определение 2.1. Слоистой сетью $\mathcal{N}_l$ для сети $\mathcal{N} = (G = (V, E), c, s, t)$ будем называть $\mathcal{N}_l = (G_l = (V, E_l), c_l, s, t)$, где

$$E_l = \{e = (u, v) \in E \mid \delta(v) - \delta(u) = 1\}$$

$$c_l(u, v) = c(u, v) \cdot \mathcal{I}\{(u, v) \in E_l\}$$

Где $\delta(v)$ — это минимальное расстояние (по количеству рёбер) от вершины s до вершины v , а $\mathcal{I}$ — функция-индикатор, которая принимает значение 1, если выражение истинно, и 0, если ложно.

На рабоче-крестьянском языке: представьте, что вы запустили BFS от вершины s . Вершины разобьются по слоям — в одном слое будут находиться вершины на одинаковом расстоянии δ от s . Оставим рёбра только между слоями, а внутри слоёв удалим. Получившаяся структура и есть слоистая сеть.

Замечание 2.2. Математически слоистую сеть можно построить для любого графа. Однако в алгоритме Диница мы всегда будем строить слоистую сеть на основе текущей остаточной сети $\mathcal{N}_f$.

Определение 2.3. *Блокирующим потоком* будем называть такой поток f в данной сети $\mathcal{N}$, что любой путь $s \rightsquigarrow t$ содержит насыщенное этим потоком ребро. Другими словами, в данной сети не найдётся такого пути из истока в сток, вдоль которого можно беспрепятственно пропустить ненулевой поток.

Замечание 2.4. Максимальный поток является блокирующим, но не всякий блокирующий поток является максимальным.

2.1 Алгоритмы поиска блокирующего потока

Постановка задачи. Дана ациклическая сеть $\mathcal{N} = (G = (V, E), c, s, t)$, требуется найти какой-нибудь блокирующий поток f в ней.

Замечание 2.5. Дальнейшие алгоритмы предназначены только для ациклических сетей, поэтому мы потребовали этого от исходной сети.

Алгоритм 2.6 (жадный поиск). Будем находить пути из s в t , пока это возможно, и пропускать по ним поток. Используя DFS или BFS, каждый путь находится за время $O(|E|)$. Поскольку каждый найденный путь насыщает как минимум одно ребро, всего будет $O(|E|)$ путей. Итоговая сложность алгоритма $O(|E|^2)$.

Алгоритм 2.7 (удаляющий обход). Аналогично жадному алгоритму, будем искать пути из s в t , но будем попутно удалять те рёбра, вдоль которых не получится дойти до стока t (следовательно, и пропустить ненулевой поток). Будем удалять ребро после того, как мы просмотрели его в обходе в глубину (кроме того случая, когда мы прошли вдоль ребра и нашли путь до стока).

Псевдокод. Для большей наглядности алгоритма приведём код.

```

1 vector<vector<Edge>> g;
2 vector<int> ptr;
3 int s, t, n;
4
5 int dfs(int u, int flow) {
6     if (flow == 0) return 0;
7     if (u == t) return flow;
8     for (int& i = ptr[u]; i < g[u].size(); ++i) {
9         Edge& e = g[u][i];
10        int v = e.to;
11        if (e.cap - e.flow > 0) {
12            int pushed = dfs(v, min(flow, e.cap - e.flow));
13            if (pushed > 0) {
14                e.flow += pushed;
15                g[v][e.rev].flow -= pushed;
16                return pushed;
17            }
18        }
19    }
20    return 0;
21 }
22
23 int main() {
24     //...
25     int total_flow = 0;

```

```

26 ptr.assign(n, 0);
27 while (true) {
28     int pushed = dfs(s, INF);
29     if (pushed == 0) break;
30     total_flow += pushed;
31 }
32 cout << total_flow << '\n';
33 }

```

Теорема 2.8 (время работы). Удаляющий обход работает за время $O(|V||E|)$.

Доказательство. Так как граф ациклический, длина любого простого пути от истока до стока не превышает $|V|$. При каждом вызове DFS мы идём вперёд (максимум $|V|$ шагов). Далее происходит одно из двух:

1. Мы дошли до стока и пропустили поток. При этом хотя бы одно ребро на пути насыщается (у него `cap - flow == 0`).
2. Мы зашли в тупик (вернулся 0). Тогда цикл делает `ptr[u]++`. Ребро удаляется из рассмотрения.

Каждое ребро может быть насыщено или удалено суммарно 1 раз. Всего рёбер $|E|$. На каждое такое удаление или насыщение мы тратим $O(|V|)$ действий (потому что длина пути не больше $|V|$). Итоговая сложность алгоритма $O(|V||E|)$. $\square$

Алгоритм 2.9 (Малхотры — Кумара — Махешвари). В рамках курса подробно не рассматривается. Отметим лишь, что алгоритм работает за $O(|V|^2)$.

2.2 Алгоритм Диница

Алгоритм 2.10 (Диница). Пусть дана сеть $\mathcal{N} = (G = (V, E), c, s, t)$. Алгоритм Диница находит максимальный поток в этой сети. Схема алгоритма:

1. Положим изначальный поток в сети $f = 0$;
2. Строим остаточную сеть $\mathcal{N}_f$;
3. Строим слоистую сеть $\mathcal{N}_{f_l}$ из остаточной сети;
4. Если в $\mathcal{N}_{f_l}$ вершина t недостижима из s , то алгоритм окончен. Иначе найдём блокирующий поток f' в $\mathcal{N}_{f_l}$;
5. Обновляем поток $f \leftarrow f + f'$, обновляем остаточную сеть вдоль пропущенного потока;
6. Повторять шаги 3-5 до завершения алгоритма.

Псевдокод. Приведём реализацию алгоритма Диница с использованием удаляющего обхода.

```

1 const int INF = 2e9;
2
3 struct Dinic {
4     struct Edge {
5         int to, rev, cap, flow;
6         Edge(int to, int rev, int cap, int flow) :
7             to(to), rev(rev), cap(cap), flow(flow) {}
8     };
9
10    int n;

```

```

11  vector<vector<Edge>> g;
12  vector<int> level;
13  vector<int> ptr;
14
15  Dinic(int n) : n(n), g(n), level(n), ptr(n) {}
16
17  void add_edge(int u, int v, int cap) {
18      g[u].emplace_back(v, (int)g[v].size(), cap, 0);
19      g[v].emplace_back(u, (int)g[u].size() - 1, 0, 0);
20  }
21
22  bool bfs(int s, int t) {
23      fill(level.begin(), level.end(), -1);
24      level[s] = 0;
25      queue<int> q;
26      q.emplace(s);
27      while (!q.empty()) {
28          int u = q.front();
29          q.pop();
30          for (int i = 0; i < g[u].size(); ++i) {
31              Edge& e = g[u][i];
32              if (e.cap - e.flow > 0 && level[e.to] == -1) {
33                  level[e.to] = level[u] + 1;
34                  q.emplace(e.to);
35              }
36          }
37      }
38      return level[t] != -1;
39  }
40
41  int dfs(int u, int t, int pushed) {
42      if (pushed == 0) return 0;
43      if (u == t) return pushed;
44      for (int& i = ptr[u]; i < g[u].size(); ++i) {
45          Edge& e = g[u][i];
46          int v = e.to;
47          if (level[u] + 1 != level[v] || e.cap - e.flow == 0) continue;
48          int push = dfs(v, t, min(pushed, e.cap - e.flow));
49          if (push == 0) continue;
50          e.flow += push;
51          g[v][e.rev].flow -= push;
52          return push;
53      }
54      return 0;
55  }
56
57  int max_flow(int s, int t) {
58      int ans_flow = 0;
59      while (bfs(s, t)) {
60          fill(ptr.begin(), ptr.end(), 0);
61          while (int pushed = dfs(s, t, INF)) {
62              ans_flow += pushed;
63          }
64      }
65      return ans_flow;
66  }

```

67 };

Теорема 2.11 (корректность). Алгоритм Диница корректно находит максимальный поток в исходной сети.

Доказательство. Действительно, алгоритм останавливается, когда сток t недостижим из истока s в слоистой сети. Но слоистая сеть — это результат BFS по остаточной сети. Если BFS не смог дойти до t , значит, пути от s до t в остаточной сети не существует. По теореме Форда — Фалкерсона, если в остаточной сети нет пути $s \rightsquigarrow t$, то поток максимален. Следовательно, алгоритм останавливается, когда поток стал максимальным. $\square$

Теорема 2.12 (время работы). Алгоритм Диница с удаляющим обходом работает за $O(|V|^2|E|)$.

Доказательство. На каждой фазе алгоритма (построение слоистой сети и поиск блокирующего потока) кратчайшее расстояние от s до t строго увеличивается. Это следует из определения блокирующего потока. Так как максимальная длина простого пути в графе не может превышать количество вершин $|V|$, алгоритм совершит не более $|V|$ фаз. Каждая фаза — удаляющий обход, который работает за $O(|V||E|)$. Итоговое время работы — $O(|V|^2|E|)$. $\square$

3 Теоремы Карзанова

Определение 3.1. Пусть дана сеть $\mathcal{N} = (G = (V, E), c, s, t)$ и $c : E \rightarrow \mathbb{Z}_{\geq 0}$. Пусть

$$c^+(v) = \sum_{u \in V} c(u, v), \quad c^-(v) = \sum_{u \in V} c(v, u)$$

Определим *потенциал вершины* как $\varphi(v) = \min(c^+(v), c^-(v))$.

На рабоче-крестьянском языке: потенциал вершины — это сколько максимум в теории через неё может протекать.

Определение 3.2. Потенциалом сети $\mathcal{N}$ с целочисленными пропускными способностями назовём

$$\varphi(\mathcal{N}) = \sum_{v \in V \setminus \{s, t\}} \varphi(v)$$

Теорема 3.3 (первая теорема Карзанова). Число итераций в алгоритме Диница в сети $\mathcal{N}$ с целочисленными пропускными способностями равно $O(\sqrt{\varphi(\mathcal{N})})$.

Доказательство. Для доказательства теоремы докажем сначала две подготовительные леммы.

Лемма 3.4. Пусть l — расстояние между s и t в исходной сети, а максимальный поток в этой сети равен F . Тогда $l \leq \frac{\varphi(\mathcal{N})}{F} + 1$.

Доказательство. Действительно, пусть V_i — это множество вершин, удалённых от s на $i \leq l$ (то есть это слой номер i при обходе BFS). При удалении V_i путь из s в t пропадёт. Тогда через V_i проходит некоторый поток f величиной $|f|$. Но через любую вершину $v \in V$ проходит не более её потенциала, то есть $\varphi(v)$ потока. Пусть $\varphi_i = \sum_{v \in V_i} \varphi(v)$, тогда $|f| \leq \varphi_i$. Кроме того, для любого $i \leq l$ выполнено $F \leq \varphi_i$ и

$$\begin{cases} F \leq \varphi_1 \\ F \leq \varphi_2 \\ \vdots \\ F \leq \varphi_{l-1} \end{cases} \implies (l-1)F \leq \sum_{i=1}^{l-1} \varphi_i \stackrel{\text{def}}{\leq} \varphi(\mathcal{N})$$

Откуда $l \leq \frac{\varphi(\mathcal{N})}{F} + 1$ — что и требовалось доказать. $\square$

Лемма 3.5. Пусть $\mathcal{N}$ — сеть, а f — поток в этой сети. Тогда $\varphi(\mathcal{N}) = \varphi(\mathcal{N}_f)$.

Доказательство. Покажем, что для произвольной вершины v (кроме истока s и стока t) значения $c^+(v)$ и $c^-(v)$ в остаточной сети остаются такими же, какими были до пропуска потока.

Действительно, фиксируем произвольную вершину $v \in V \setminus \{s, t\}$. Предположим, мы пустили поток величины x по пути, проходящему через вершину v . Значит, этот поток вошёл в v по какому-то ребру (u, v) и вышел из v по какому-то ребру (v, w) . Рассмотрим, как изменились пропускные способности в остаточной сети $\mathcal{N}_f$ для вершины v : 1. (u, v) . Прямая пропускная способность $c(u, v)$ уменьшилась на x , следовательно, $c^+(v)$ уменьшилась на x . Обратная пропускная способность $c(v, u)$ увеличилась на x . Следовательно, $c^-(v)$ увеличилась на x . 2. (v, w) . Прямая пропускная способность $c(v, w)$ уменьшилась на x . Значит, $c^-(v)$ уменьшилась на x . Обратная пропускная способность $c(w, v)$ увеличилась на x . Следовательно, $c^+(v)$ увеличилась на x .

Итого имеем

$$\begin{aligned} c^+(v)_{new} &= c^+(v)_{old} - x + x = c^+(v)_{old} \\ c^-(v)_{new} &= c^-(v)_{old} + x - x = c^-(v)_{old} \end{aligned}$$

Таким образом, $c^+(v)$ и $c^-(v)$ не изменились, а значит, потенциал $\varphi(v)$ не изменился, следовательно, и $\varphi(\mathcal{N})$ остался прежним — что и требовалось доказать. $\square$

Докажем теперь первую теорему Карзанова. Пусть F — максимальный поток, который можно пропустить в этой сети. В процессе работы алгоритма мы постепенно увеличиваем текущий поток $|f|$. Значит, в остаточной сети нам осталось протолкнуть поток величиной $F_{res} = F - |f|$. Разделим весь процесс работы алгоритма Диница на две части, в зависимости от того, сколько потока нам осталось протолкнуть. Границей между частями будет значение $\sqrt{\varphi(\mathcal{N})}$: 1. $F - |f| \leq \sqrt{\varphi(\mathcal{N})}$. Так как пропускные способности у нас целочисленные, на каждой фазе алгоритма Диница мы проталкиваем хотя бы 1 единицу потока. Значит, чтобы протолкнуть оставшийся поток $F - |f|$, потребуется не более $\sqrt{\varphi(\mathcal{N})}$ фаз. Итого число итераций в этом случае равно $O(\sqrt{\varphi(\mathcal{N})})$. 2. $F - |f| \geq \sqrt{\varphi(\mathcal{N})}$. Рассмотрим самую последнюю фазу этого этапа (фаза X). Применим к остаточной сети перед фазой X две доказанные леммы. По второй лемме потенциал текущей остаточной сети равен $\varphi(\mathcal{N})$. По первой лемме расстояние l от s до t ограничено формулой: $l \leq \frac{\varphi(\mathcal{N})}{F - |f|} + 1$. Имеем:

$$\begin{cases} l \leq \frac{\varphi(\mathcal{N})}{F - |f|} + 1 \\ F - |f| \geq \sqrt{\varphi(\mathcal{N})} \end{cases} \implies l \leq \sqrt{\varphi(\mathcal{N})} + 1$$

Это значит, что на момент фазы X длина кратчайшего пути l в остаточной сети не превышает $\sqrt{\varphi(\mathcal{N})} + 1$. Но поскольку в алгоритме Диница на каждой фазе длина пути l увеличивается хотя бы на 1, с начала алгоритма в этом случае пройдёт не более $\sqrt{\varphi(\mathcal{N})} + 1$ фаз.

Таким образом, всего произойдёт $O(\sqrt{\varphi(\mathcal{N})})$ фаз алгоритма Диница. $\square$

Теорема 3.6 (вторая теорема Карзанова, б/д). Число итераций поиска блокирующего потока в схеме Диница с целочисленными пропускными способностями равно $O(C^{\frac{1}{3}}|V|^{\frac{2}{3}})$, где C — максимальная пропускная способность ребра.

Доказательство. [Доказательство](#) $\square$

4 Алгоритм Хопкрофта — Карпа

Постановка задачи. Дан двудольный граф $G = (V = L \sqcup R, E)$, требуется найти максимальное паросочетание в нём.

Алгоритм 4.1 (Хопкрофта — Карпа). Зададим сеть $\mathcal{N} = (G = (V' = L \sqcup R \sqcup \{s, t\}, E'), c, s, t)$, где $c : E' \rightarrow \{1\}$. Из s проведём рёбра во все вершины L и рёбра из всех вершин R в вершину t . Ориентируем все рёбра между L и R от вершин из L к вершинам из R . Запустим алгоритм Диница на полученной сети. Рёбра между L и R , по которым пройдёт поток, будут образовывать искомое максимальное паросочетание.

Теорема 4.2 (время работы). Алгоритм Хопкрофта — Карпа работает за $O(|E| \sqrt{|V|})$.

Доказательство. В построенной сети для каждой вершины $v \in V' \setminus \{s, t\}$ её потенциал не превышает 1 (он равен 1, если вершина не изолирована). Следовательно, $\varphi(\mathcal{N}) \leq |V'| - 2 = |V|$. Кроме того, у удаляющего обхода в единичной сети есть следующее свойство: - Если поиск пути оказался неуспешным, то ребро, по которому мы пришли в тупик, сразу удаляется; - Если путь оказался успешным и мы пришли в t , мы пропускаем единичный поток, и все рёбра найденного пути сразу удаляются (так как они единичные и сразу насыщаются).

Таким образом, каждый раз, когда DFS проходит по ребру (вперёд или откатываясь назад), это ребро вскоре удаляется и больше на этой фазе не рассматривается. Следовательно, на работу с рёбрами на каждой фазе мы тратим $O(|E|)$ времени. А всего итераций по первой теореме Карзанова $O(\sqrt{\varphi(\mathcal{N})}) = O(\sqrt{|V|})$. Перемножая, получаем итоговую асимптотику $O(|E| \sqrt{|V|})$. $\square$

Замечание 4.3. Алгоритм Куна также решает задачу поиска максимального паросочетания в двудольном графе. Причём он работает за $O(|V||E|)$, а алгоритм Хопкрофта — Карпа за $O(|E| \sqrt{|V|})$, то есть в $\sqrt{|V|}$ раз быстрее.